1 0 8

# fixtures/evidence_ref/ — T self-validation fixtures

Fixture convention for `constitution/scripts/fastcycle/context/evidence_ref.py` (contract `specs/004-fast-dev-cycles/contracts/evidence-reference-reverify.md`, clauses ER-000..ER-005; data-model.md §8 "Evidence Reference"; plan.md T-E05; research.md DEC-29). Task T108 ( `tasks.md` ) writes ONLY this RED test + these fixtures; the tool itself is T117, a later, separate task (Producer != Verifier, §11.4.240 — this file's author never implements `evidence_ref.py` ).

## Layout

Six self-contained scenario directories, one per contract RED-fixtures-table row. Each holds the CURRENT ("consume-time"/live) bytes of every file the scenario's `ref.json` cites, plus (only where the scenario's whole point is that a file changed after the reference was created) an `*_as_recorded.*` sibling holding the ORIGINAL ("create-time") bytes that `ref.json` 's stored content addresses were computed from. `ref.json` 's declared `inputs.paths` never change between create and consume — only the BYTES at those paths change, which is exactly what ER-002 ("recomputes every component from current bytes") exists to catch.

| Directory | Kind | What changed between create and consume |
|---|---|---|
| `er_unchanged/` | golden-good | nothing — `input_a.txt`, `verifier.sh`, `evidence.txt` are byte-identical to what `ref.json` records |
| `er_changed_input/` | golden-bad | `input_a.txt`'s CURRENT bytes differ from `input_a_as_recorded.txt` (what `ref.json`'s `inputs.root` was computed from) |
| `er_changed_verifier/` | golden-bad | `verifier.sh`'s CURRENT bytes differ from `verifier_as_recorded.sh` (what `ref.json`'s `verifier_version` was computed from); the declared input is unaffected |
| `er_changed_target/` | golden-bad | nothing on disk — `ref.json` records `target_fingerprint: "DEVICE-SERIAL-AAA"`; `scenario.json` names the DIFFERENT live serial (`DEVICE-SERIAL-BBB`) `consume` is invoked with — T108's own named scenario, "a reference whose on-target fingerprint differs is refused" |
| `er_evidence_tampered/` | golden-bad | `evidence.txt`'s CURRENT bytes differ from `evidence_as_recorded.txt` (what `ref.json`'s `evidence.content_address` was computed from); inputs + verifier unaffected |
| `er_negctrl_unrelated_file/` | negative-control | `unrelated.txt`'s CURRENT bytes differ from `unrelated_as_recorded.txt` — but `unrelated.txt` is NEVER named in `ref.json`'s `inputs.paths`, so ER-002 must never notice it |

Every sha256 embedded in every `ref.json` is the REAL digest of the file it names, computed at fixture-authoring time by `hashlib.sha256` and independently RE-VERIFIED, live, by the already-landed `constitution/scripts/fastcycle/lib/fc_common.py body-hash --verify` (a control needle: the established tool, not this fixture author's own reimplementation, confirms every `ref.json`'s stored `body_hash` matches its canonical body — `bash test_evidence_ref_red.sh` reruns this check on every invocation, so a future hand-edit that desyncs a `ref.json` from its own `body_hash` is caught rather than silently poisoning the fixture).

[117]

## `ref.json` / `consume.json` wire format (UNCONFIRMED, binding-if-adopted on T )

The contract's Invocation section names the CLI shape ( `create --fact ... --verifier <id> --inputs <paths...> [--target <serial>] --verdict PASS|FAIL --evidence <path> --out <ref.json>` ; `consume --ref <ref.json> [--target <serial>] --out <consume.json>` ) and data-model.md §8 names the Evidence Reference entity's FIELDS, but neither pins the exact JSON shape those fields are serialized into, nor how `--verifier <id>` resolves to the bytes ER-001's `verifier_version` hashes. This fixture set — like the established precedent in `fixtures/verdict_cache/` and `test_governance_subset_red.sh`'s own "UNCONFIRMED... DEFINED here" section — DEFINES one concrete, defensible shape and flags it explicitly as binding-if-adopted, not as a claim the contract already specifies it:

- **`--verifier <id>` IS the verifier script's own repo-relative path** (the simplest literal reading available — the contract offers no separate `--verifier-script` flag to distinguish an opaque id from a locator). `verifier_version` = `content_address(that path's bytes at create time)` .
- **`ref.json` carries BOTH the Merkle root AND the declared path list** ( `inputs: {paths: [...], root: "sha256:..."}` ), never the root alone — a `consume` cannot recompute a Merkle root over an unknown path set, so the paths must travel with the reference.

- **Content addresses and Merkle roots are `"sha256:<64 hex>"`-prefixed** (data-model.md §0's own `ContentAddress` / `MerkleRoot` format); `body_hash` is the established `fc_common.py` convention's BARE 64-hex string (no prefix) — two different, independently-real established formats, each used exactly where its own convention already defines it, never conflated.
- **`changed_components` is a closed vocabulary**: `"inputs:<path>"` (that declared input's content changed), `"verifier_version"`, `"target_fingerprint"` (the live `--target` differs from the recorded one), and `"target_fingerprint_unobserved"` (ER-005: `--target` omitted for a non-hermetic reference) — DEFINED here; the contract names none of these literally, only "listing the changed components."
- **Merkle root canonicalization** reuses `fc_common.py`'s own `canon()` convention ( `json.dumps(sort_keys=True, separators=(",",":"), ensure_ascii=False)` ) over the sorted `[path, content_address]` pair list — never a bespoke separator scheme — per §11.4.227 (extend, don't invent a second serialization convention alongside an already-established one). Pairs sort by the path string's raw UTF-8 BYTES (data-model.md §0's own "sorted bytewise by path"), never a locale-dependent string sort (C-003; the T105 GNU-`sort`-vs-Python lesson applies here too, though every path in this fixture set is plain ASCII so the two orderings happen to coincide — the bytewise construction is used anyway, on principle, not by luck).

## What this RED test does NOT cover (honest, explicit gap)

ER-004 ("a ref whose verifier is an unrecorded LLM judgment is refused at `create`") and exit code 4 ("target unreadable") are real contract clauses but are NOT among the six RED-fixtures-table rows this directory encodes — T108's own task line names only the two-step (unchanged/changed) `consume` fixture plus the named golden-bad target-mismatch scenario, and the contract's fixture table itself has no row for either. Left for T117 review time, exactly as `fixtures/governance_subset/`'s own `--item` resolution gap was left for T112.